TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
938 Aurora Blvd., Cubao, Quezon City

PROGRAMMING LOGIC & DESIGN
CPE 007 | CPE11S2

FINAL PROJECT:
TEXT-BASED EDUCATIONAL GAME

Submitted by:

AGUILAR, SOPHIA L.

CASTRO, RAFSIL F.

MALBAS, SEAN GILMAR P.

POLICINA, MA. ANDREA S.

TABAOG, ALTHEA B.

Submitted to:

ENGR. NOEL JASON LUSUNG

Subject Professor

INTRODUCTION

This final project in Programming Logic and Design presents *The Last Trial: Secrets of Blackwood Academy*, a console-based educational adventure game developed using C++ to demonstrate the application of fundamental programming concepts through interactive gameplay. Set within a mysterious academy, the game allows players to navigate interconnected rooms through text-based commands, discover clues, collect a key and special artifacts, and answer multiple-choice questions in English, Mathematics, Science, and Computer subjects. Players must defeat four subject-specific guardians and obtain the four Seals of Knowledge to unlock the Sealed Chamber, where they face the Headmaster and complete the final trial. Correct answers award experience points and reduce an opponent's health, while incorrect answers decrease the player's health. The system also tracks inventory, collected seals, defeated guardians, and rank progression. By integrating variables, structures, arrays, conditional statements, loops, and functions, the project applies programming logic to command processing, game-state management, and the progression from exploration to victory or game over.

The program also uses file streams to save and restore player progress. It supports manual saving, automatic saving attempts, and validation of loaded data. After defeat, the player may select START OVER and enter the same name to restore health while retaining accumulated progress.

STATEMENT OF THE PROBLEM

This project aims to develop The Last Trial: Secrets of Blackwood Academy, an interactive, console-based educational adventure game using C++. It addresses the design and implementation of a system that processes text commands, presents educational challenges, manages player information, and controls gameplay progression.

Specifically, the project seeks to answer the following questions:

1. How can the system enable players to navigate the interconnected rooms of Blackwood Academy using text-based commands?

2. How can the system recognize and process player commands for exploration, interaction, and information display while handling invalid or empty inputs?

3. How can multiple-choice questions in English, Mathematics, Science, and Computer subjects be integrated into guardian battles and the final trial?

4. How can the system apply experience-point rewards, opponent damage, and player-health penalties based on the outcomes of educational challenges?

5. How can the system track and update player information, including health, experience points, rank, inventory, collected seals, and defeated guardians?

6. How can the system save player progress, validate stored data, restore a saved game, and handle file-operation errors?

OBJECTIVES

The general objective is to develop The Last Trial: Secrets of Blackwood Academy, a console-based educational adventure game that applies fundamental C++ programming concepts.

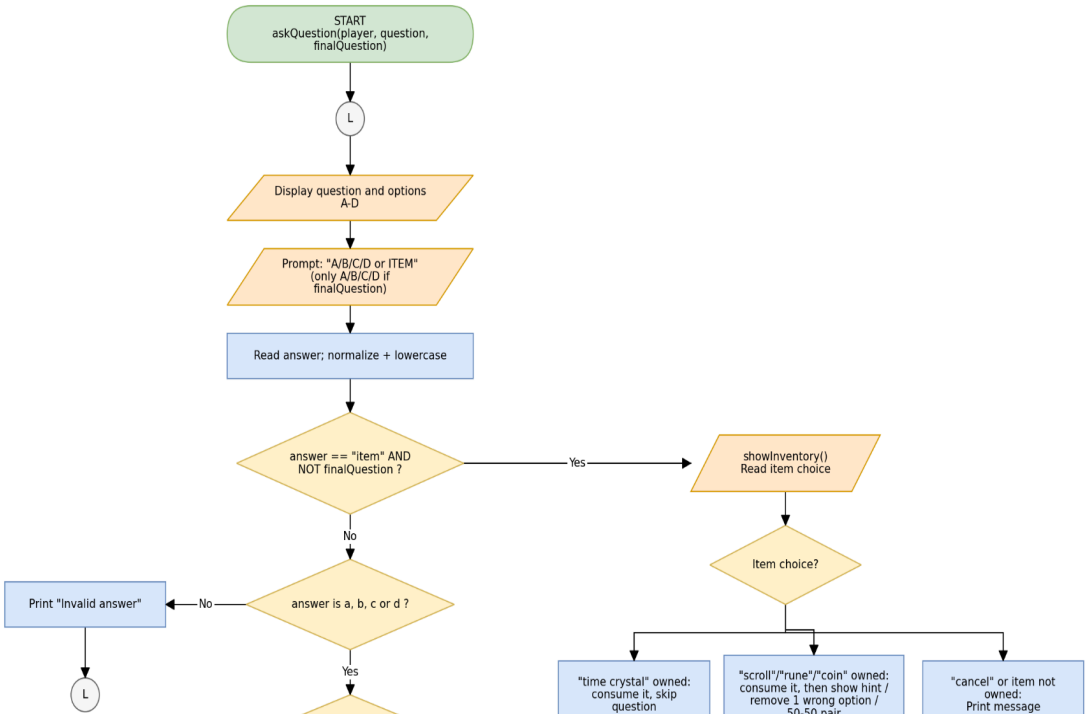
Specifically, the project aims to:

- Implement navigation through the interconnected rooms of Blackwood Academy using text-based commands.
- Process commands for game actions and information display while handling invalid or empty commands.
- Integrate multiple-choice questions in English, Mathematics, Science, and Computer subjects into guardian battles and the final trial.
- Award experience points, reduce opponent health, and deduct player health according to challenge outcomes.
- Track and update player health, experience points, rank, inventory, collected seals, and defeated guardians.
- Implement manual saving, automatic saving attempts, and validated loading of player progress using file streams.

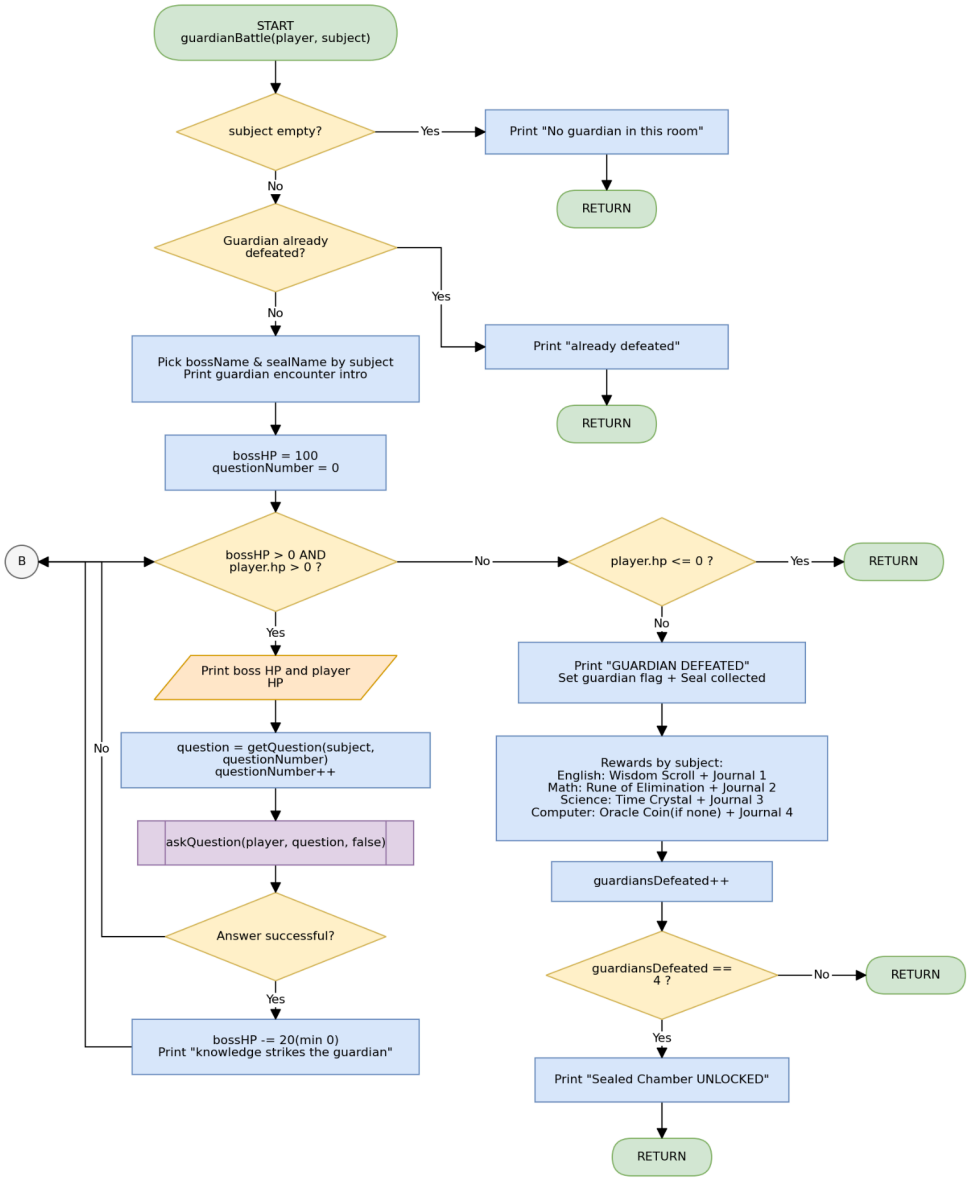
FLOWCHART OF THE SYSTEM

I. Headmaster Final Battle

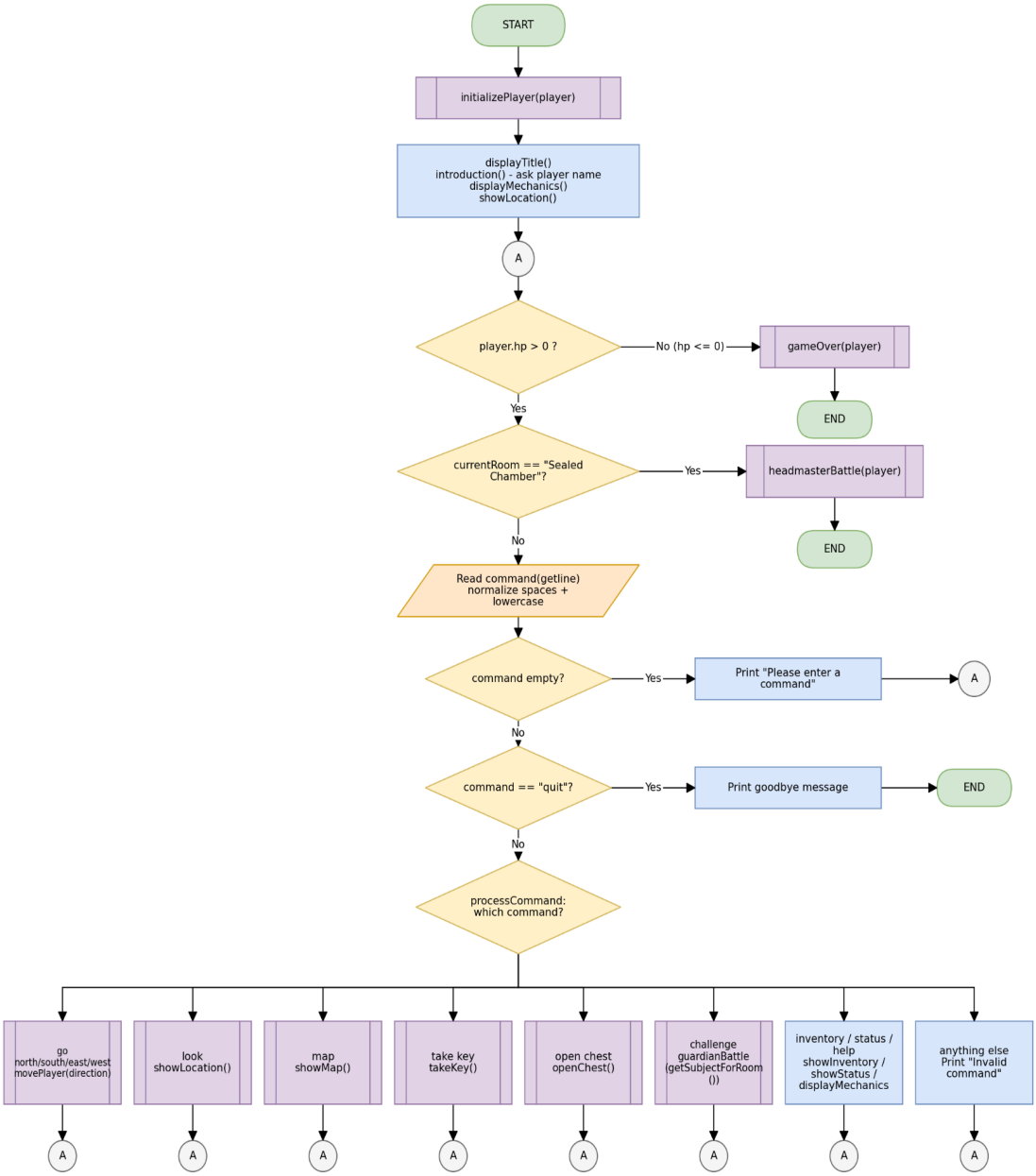
II. Ask Questions (Battle Question)



III. Guardian Battle



IV. Main Game Flow



PSEUDOCODE

THE LAST TRIAL - SECRETS OF BLACKWOOD ACADEMY

PSEUDOCODE

==========

DATA

Player:

name, currentRoom, rank

hp, xp, correct, incorrect, guardiansDefeated

hasKey, libraryKeyTaken, chestOpened

items: hasTimeCrystal, hasRuneOfElimination, hasOracleCoin, hasWisdomScroll

seals: sealLanguage, sealLogic, sealDiscovery, sealTechnology

defeated flags: archivistDefeated, numeralDefeated, vexDefeated, glitchDefeated

journal1..journal4

Question:

text, optionA..optionD, answer (a/b/c/d), hint

ROOMS (map)

Main Hall: N -> Laboratory, S -> Server Room, E -> Hall of Numbers, W -> Library

Library, Hall of Numbers, Laboratory: only exit back to Main Hall

Server Room: N -> Main Hall, S -> Sealed Chamber (only if guardiansDefeated == 4)

Sealed Chamber: N -> Server Room (entering it starts the final battle)

ROOM -> SUBJECT -> GUARDIAN

Library -> English -> The Archivist -> Seal of Language

Hall of Numbers -> Mathematics -> The Numeral -> Seal of Logic

Laboratory -> Science -> Dr. Vex -> Seal of Discovery

Server Room -> Computer -> The Glitch -> Seal of Technology

MAIN

BEGIN main

CREATE player

initializePlayer(player)

displayTitle()

introduction(player)

displayMechanics()

showLocation(player)

gameLoop(player)

END

initializePlayer(player)

name = ""; currentRoom = "Main Hall"; rank = "Beginner"

hp = 100; xp = 0

correct = incorrect = guardiansDefeated = 0

all item / seal / defeated / journal / key / chest flags = false

```
introduction(player)
    PROMPT "Enter player name"
    READ name
    WHILE name is empty after normalizeSpaces
        PROMPT "Please enter a valid name"
        READ name
    DISPLAY prologue story and Four Seals objective



toLowerCase(text)       -> return text with every character lowercase
normalizeSpaces(text)   -> trim, collapse repeated whitespace into a single space



GAME LOOP
---------
gameLoop(player)
    WHILE player.hp > 0
        IF currentRoom == "Sealed Chamber" THEN
            headmasterBattle(player)
            RETURN
        DISPLAY room name and prompt
        READ command (if input ends, RETURN)
        command = toLowerCase(normalizeSpaces(command))
        IF command is empty THEN
            DISPLAY "Please enter a command"
            CONTINUE
```

```
        IF command == "quit" THEN
            DISPLAY goodbye message
            RETURN
        processCommand(player, command)
    END WHILE
    IF player.hp <= 0 THEN gameOver(player)



processCommand(player, command)
    IF command is "go north" / "go south" / "go east" / "go west"
        movePlayer(player, direction part of command)
    ELSE IF command == "look"       THEN showLocation(player)
    ELSE IF command == "map"        THEN showMap(player)
    ELSE IF command == "take key"   THEN takeKey(player)
    ELSE IF command == "open chest" THEN openChest(player)
    ELSE IF command == "challenge"  THEN guardianBattle(player, getSubjectForRoom(player))
    ELSE IF command == "inventory"  THEN showInventory(player)
    ELSE IF command == "status"     THEN showStatus(player)
    ELSE IF command == "help"       THEN displayMechanics()
    ELSE DISPLAY "Invalid command"



EXPLORATION
-----------
movePlayer(player, direction)
    nextRoom = ""
```

```
    LOOK UP nextRoom from the map table using (currentRoom, direction)
    SPECIAL CASE: currentRoom == "Server Room" AND direction == "south"
        IF guardiansDefeated == 4 THEN nextRoom = "Sealed Chamber"
        ELSE
            DISPLAY locked door message (guardiansDefeated of 4 symbols glowing)
            RETURN
    IF nextRoom is empty THEN
        DISPLAY "You cannot go <direction> from <room>"
        RETURN
    currentRoom = nextRoom
    DISPLAY movement text
    showLocation(player)



showLocation(player)
    DISPLAY description for currentRoom
    IF Library:
        IF NOT libraryKeyTaken THEN hint to "take key"
        IF NOT archivistDefeated THEN show Archivist intro ELSE show "quiet"
    IF Hall of Numbers:
        IF NOT chestOpened THEN hint to "open chest"
        IF NOT numeralDefeated THEN show Numeral intro ELSE show "clocks stopped"
    IF Laboratory:   IF NOT vexDefeated    THEN show Dr. Vex intro
    IF Server Room:  IF NOT glitchDefeated THEN show Glitch intro
                     DISPLAY whether Sealed Chamber is LOCKED / UNLOCKED
    IF Sealed Chamber: DISPLAY hidden staircase text
```

```
showMap(player)
    COPY ASCII floor plan
    SET (row, col) according to currentRoom
    PUT '*' at (row, col)
    DISPLAY map, location, guardiansDefeated/4, Sealed Chamber LOCKED/UNLOCKED



takeKey(player)
    IF currentRoom != "Library"  THEN DISPLAY "no key here"; RETURN
    IF libraryKeyTaken           THEN DISPLAY "already collected"; RETURN
    libraryKeyTaken = true
    hasKey = true
    DISPLAY "MYSTERIOUS KEY ACQUIRED"



openChest(player)
    IF currentRoom != "Hall of Numbers" THEN DISPLAY "no chest here"; RETURN
    IF chestOpened                   THEN DISPLAY "already open"; RETURN
    IF NOT hasKey                     THEN DISPLAY "locked, need a key"; RETURN
    hasKey = false
    chestOpened = true
    hasOracleCoin = true
    DISPLAY "ORACLE'S COIN ACQUIRED"
```

```
showInventory(player)
    FOR each owned item (key, Time Crystal, Rune, Oracle Coin, Wisdom Scroll)
        DISPLAY name and effect
    IF no items THEN DISPLAY "Your inventory is empty. "
    FOR each seal DISPLAY COLLECTED or MISSING


showStatus(player)
    DISPLAY name, room, hp/100, xp, rank, correct, incorrect, guardiansDefeated/4


updateRank(player)
    oldRank = rank
    IF xp >= 300 THEN rank = "Master"
    ELSE IF xp >= 200 THEN rank = "Scholar"
    ELSE IF xp >= 100 THEN rank = "Apprentice"
    ELSE rank = "Beginner"
    IF rank != oldRank THEN DISPLAY "RANK EVOLUTION: oldRank -> rank"


getSubjectForRoom(player)
    Library -> "English"; Hall of Numbers -> "Mathematics"
    Laboratory -> "Science"; Server Room -> "Computer"
    otherwise -> ""
```

QUESTIONS

---

getQuestion(subject, number)

    number = number MOD 5

    Each subject (English, Mathematics, Science, Computer) holds 5 questions

    RETURN question[number] of the chosen subject (any other subject -> Computer)

askQuestion(player, question, finalQuestion)

    LOOP forever

        DISPLAY question and options A-D

        IF finalQuestion THEN PROMPT "A/B/C/D" ELSE PROMPT "A/B/C/D or ITEM"

        READ answer; answer = toLowerCase(normalizeSpaces(answer))

        IF answer == "item" AND NOT finalQuestion THEN

            showInventory(player)

            READ item choice

            IF item == "time crystal" AND hasTimeCrystal

                hasTimeCrystal = false

                DISPLAY "question skipped"

                RETURN true   // no damage, counts as success

            ELSE IF item == "scroll" AND hasWisdomScroll

                hasWisdomScroll = false

                DISPLAY question.hint

                CONTINUE LOOP

```
    ELSE IF item == "rune" AND hasRuneOfElimination
        hasRuneOfElimination = false
        DISPLAY the first wrong option as INCORRECT
        CONTINUE LOOP
    ELSE IF item == "coin" AND hasOracleCoin
        hasOracleCoin = false
        DISPLAY "correct answer is <correct letter> or <first wrong letter>"
        CONTINUE LOOP
    ELSE IF item == "cancel"
        DISPLAY "no artifact used"; CONTINUE LOOP
    ELSE
        DISPLAY "You do not have that artifact"; CONTINUE LOOP


IF answer is a, b, c or d THEN
    IF answer == question.answer THEN
        correct = correct + 1
        xp = xp + 20
        DISPLAY CORRECT, +20 XP
        updateRank(player)
        RETURN true
    ELSE
        incorrect = incorrect + 1
        hp = hp - 20 (minimum 0)
        DISPLAY WRONG, correct letter, current HP
        RETURN false
```

```
        DISPLAY "Invalid answer"      // loop again


GUARDIAN BATTLE

---------------

guardianBattle(player, subject)
    IF subject is empty THEN DISPLAY "no guardian here"; RETURN
    SET bossName, sealName, alreadyDefeated from subject
    IF alreadyDefeated THEN DISPLAY "already defeated"; RETURN
    DISPLAY encounter intro for the subject


    bossHP = 100
    questionNumber = 0
    WHILE bossHP > 0 AND player.hp > 0
        DISPLAY boss HP and player HP
        question = getQuestion(subject, questionNumber)
        questionNumber = questionNumber + 1
        successful = askQuestion(player, question, false)
        IF successful THEN
            bossHP = bossHP - 20 (minimum 0)
            DISPLAY "guardian takes 20 damage"
    END WHILE


    IF player.hp <= 0 THEN RETURN          // gameLoop will show Game Over


    DISPLAY "<bossName> DEFEATED"
    CASE subject
```

```
        English:     archivistDefeated = true; sealLanguage = true
                     hasWisdomScroll = true;   discoverJournal(player, 1)
        Mathematics: numeralDefeated = true;   sealLogic = true
                     hasRuneOfElimination = true; discoverJournal(player, 2)
        Science:     vexDefeated = true;       sealDiscovery = true
                     hasTimeCrystal = true;    discoverJournal(player, 3)
        Computer:    glitchDefeated = true;    sealTechnology = true
                     IF NOT hasOracleCoin THEN hasOracleCoin = true
                     discoverJournal(player, 4)
    guardiansDefeated = guardiansDefeated + 1
    DISPLAY guardiansDefeated/4
    IF guardiansDefeated == 4 THEN
        DISPLAY "Sealed Chamber UNLOCKED - go SOUTH from the Server Room"



discoverJournal(player, page)
    SET journal<page> = true
    DISPLAY the matching journal text (pages 1-4)



FINAL BATTLE
------------
headmasterBattle(player)
    DISPLAY intro scene
    headmasterHP = 200
    questionNumber = 0
```

```
phaseTwoShown = false
phaseThreeShown = false
subjects = [English, Mathematics, Science, Computer]

WHILE headmasterHP > 20 AND player.hp > 0
    IF headmasterHP <= 100 AND NOT phaseTwoShown
        phaseTwoShown = true; DISPLAY PHASE II scene
    IF headmasterHP <= 50 AND NOT phaseThreeShown
        phaseThreeShown = true; DISPLAY PHASE III scene (truth revealed)
    DISPLAY headmaster HP and player HP
    currentSubject = subjects[questionNumber MOD 4]
    question = getQuestion(currentSubject, questionNumber)
    questionNumber = questionNumber + 1
    successful = askQuestion(player, question, false)
    IF successful THEN
        headmasterHP = headmasterHP - 20 (never below 20)
        DISPLAY "HEADMASTER -20 HP"
END WHILE

IF player.hp <= 0 THEN gameOver(player); RETURN

// Final phase: no items allowed
hasTimeCrystal = hasRuneOfElimination = hasOracleCoin = hasWisdomScroll = false
finalQuestion = "What does true knowledge require?"
    correct answer = C ("Continuing to learn even when you do not know")
```

```
    WHILE player.hp > 0
        correct = askQuestion(player, finalQuestion, true)
        IF correct THEN
            victoryEnding(player)
            RETURN
        IF player.hp > 0 THEN DISPLAY "There is no escape from the final lesson"
    END WHILE
    gameOver(player)



ENDINGS
-------
victoryEnding(player)
    DISPLAY victory story
    DISPLAY name, final rank, xp, correct, incorrect, guardiansDefeated/4, "FINAL BOSS: DEFEATED"


gameOver(player)
    DISPLAY game over story
    DISPLAY final xp and final rank
```

DATA DICTIONARY

The program uses two user-defined structures, *Player* and *Question*, to organize player information and educational questions. Additional variables support command processing, navigation, inventory display, and battles.

I. Player Structure

These fields belong to the *Playe*r structure. Their initial values are assigned by *initializePlayer()*.

Field Name	**Data Type**	**Initial Value**	**Description**
name	string	""	Stores the player's name entered through the keyboard.
currentRoom	string	"Main Hall"	Stores the player's current location.
rank	string	"Beginner"	Stores the player's rank, which is updated according to accumulated XP.
hp	int	100	Stores remaining health points. Each incorrect answer deducts 20 HP; values below zero are reset to zero.
xp	int	0	Stores experience points. Each correct answer

			adds 20 XP.
correct	int	0	Counts correctly answered questions.
incorrect	int	0	Counts incorrectly answered questions.
guardiansDefeated	int	0	Counts defeated guardians. Four defeated guardians unlock access to the Sealed Chamber.
hasKey	bool	false	Indicates whether the player currently possesses the key.
libraryKeyTaken	bool	false	Records whether the Library key has already been collected.
chestOpened	bool	false	Records whether the chest in the Hall of Numbers has already been opened.
hasTimeCrystal	bool	false	Indicates whether the player possesses a Time Crystal.

hasRuneOfElimination	bool	false	Indicates whether the player possesses a Rune of Elimination.
hasOracleCoin	bool	false	Indicates whether the player possesses an Oracle’s Coin.
hasWisdomScroll	bool	false	Indicates whether the player possesses a Wisdom Scroll.
sealLanguage	bool	false	Records whether the Seal of Language has been obtained.
sealLogic	bool	false	Records whether the Seal of Logic has been obtained.
sealDiscovery	bool	false	Records whether the Seal of Discovery has been obtained.
sealTechnology	bool	false	Records whether the Seal of Technology has been obtained.

archivistDefeated	bool	false	Records whether The Archivist has been defeated.
numeralDefeated	bool	false	Records whether The Numeral has been defeated.
vexDefeated	bool	false	Records whether Dr. Vex has been defeated.
glitchDefeated	bool	false	Records whether The Glitch has been defeated.
journal1	bool	false	Records whether the first journal page has been discovered.
journal2	bool	false	Records whether the second journal page has been discovered.
journal3	bool	false	Records whether the third journal page has been discovered.
journal4	bool	false	Records whether the fourth journal page has been discovered.

For Boolean fields, *true* means that the stated condition is satisfied, while *false* means that it is not.

II. Question Structure

The following fields belong to the *Question* structure. Their values are supplied by the question-bank entries or assigned when constructing the final question.

Field Name	**Data Type**	**Value or Content**	**Description**
text	string	Question text	Stores the question displayed to the player.
optionA	string	Option A text	Stores the first answer option.
optionB	string	Option B text	Stores the second answer option.
optionC	string	Option C text	Stores the third answer option.
optionD	string	Option D text	Stores the fourth answer option.

answer	string	"a", "b", "c", or "d"	Stores the correct answer's lowercase letter.
hint	string	Hint text	Stores the clue displayed through the Wisdom Scroll. The final question uses an empty string.

III. Arrays and Map Container

The array types below are shown as their element type followed by their declared size.

Identifier	Data Type	Scope	Description
english	Question[5]	getQuestion()	Stores five English questions, including their options, correct answers, and hints.
mathematics	Question[5]	getQuestion()	Stores five Mathematics questions, including their options, correct answers, and

			hints.
science	Question[5]	getQuestion()	Stores five Science questions, including their options, correct answers, and hints
computer	Question[5]	getQuestion()	Stores five Computer questions, including their options, correct answers, and hints.
subjects	string[4]	headmasterBattle()	Stores "English", "Mathematics", "Science", and "Computer" for the final battle's subject cycle.
view	vector<string>	showMap()	Stores the text lines used to display the academy map.

IV. Local Variables

Identifier	**Data Type**	**Scope**	**Initial Value or Assignment**	**Description**

player	Player	main()	Initialized by initializePlayer()	Holds the player's information and game progress.
result	string	normalizeSpaces()	""	Builds the normalized input string.
previousWasSpace	bool	normalizeSpaces()	true	Tracks whitespace to prevent repeated spaces and remove leading whitespace.
c	char	Loop in normalizeSpaces()	Current character in text	Represents each character examined during whitespace normalization.
command	string	gameLoop()	Keyboard input	Stores the player's command, which is normalized before processing.
row	int	showMap()	9	Stores the row index for

				the player's map marker; changes according to the current room.
col	int	showMap()	27	Stores the column index for the player's map marker; changes according to the current room.
line	const string&	Display loop in showMap()	Current element of view	References each map line while it is displayed.
nextRoom	string	movePlayer()	""	Stores the destination determined by the current room and requested direction.
empty	bool	showInventory()	true	Tracks whether any key or artifact has been displayed. Seal status is displayed separately.

oldRank	string	updateRank()	player.rank	Stores the previous rank so the program can detect and announce a rank change.
answer	string	askQuestion()	Keyboard input	Stores the player's answer or the "item" command.
item	string	Item-selection block in askQuestion()	Keyboard input	Stores the selected artifact command or "cancel".
possible	string	Rune block in askQuestion()	"abcd"	Supplies the option letters examined to identify an incorrect choice.
choice	char	Rune-processing loop	Current character in possible	Represents the option letter being checked.
current	string	Rune-processing loop	string(1, choice)	Converts the option character into a one-character string for comparison with the

				correct answer.
correct	char	Oracle’s Coin block	Uppercase form of question.answer[0]	Stores the correct option letter displayed by the Oracle’s Coin.
wrong	char	Oracle’s Coin block	'A', then the first incorrect option	Stores the incorrect option displayed alongside the correct option.
choice	char	Oracle’s Coin loop	'A'; increments through 'D' until an incorrect option is found	Represents the option letter being checked.
bossName	string	guardianBattle()	Assigned according to subject	Stores the current guardian’s name.
sealName	string	guardianBattle()	Assigned according to subject	Stores the corresponding seal’s name.

alreadyDefeated	bool	guardianBattle()	false, then the corresponding defeat flag	Determines whether the guardian has already been defeated.
bossHP	int	guardianBattle()	100	Stores the guardian's remaining health. Successful question outcomes deduct 20 HP.
questionNumber	int	guardianBattle()	0	Tracks the guardian battle's question sequence.
question	Question	Loop in guardianBattle()	Returned by getQuestion()	Stores the current guardian-battle question.
successful	bool	Loop in guardianBattle()	Returned by askQuestion()	Determines whether the guardian receives damage.
headmasterHP	int	headmasterBattle()	200	Stores the Headmaster's health during the main

				battle, which continues until it reaches 20 HP or player health reaches zero.
questionNumber	int	headmasterBattle()	0	Tracks the final battle's question sequence and determines subject selection.
phaseTwoShown	bool	headmasterBattle()	false	Prevents the Phase II narrative from being displayed more than once.
phaseThreeShown	bool	headmasterBattle()	false	Prevents the Phase III narrative from being displayed more than once.
currentSubject	string	Main loop in headmasterBattle()	subjects [questionNumber % 4]	Stores the subject selected for the current question.

question	Question	Main loop in headmasterBattle()	Returned by getQuestion()	Stores the current question in the Headmaster battle.
successful	bool	Main loop in headmasterBattle()	Returned by askQuestion()	Determines whether the Headmaster receives damage.
finalQuestion	Question	headmasterBattle()	Fields assigned individually	Stores the final question, its options, correct answer "c", and empty hint.
correct	bool	Final-question loop in headmasterBattle()	Returned by askQuestion()	Determines whether the final answer triggers the victory ending.

V. Function and Lambda Parameters

Parameter	**Data Type**	**Scope or Usage**	**Description**
player	Player&	Functions receiving a mutable player reference	References the existing player object, allowing its fields to be

			updated.
player	const Player&	showMap(), showLocation(), showInventory(), showStatus(), getSubjectForRoom(), and gameOver()	Provides access to player information without allowing modification through this parameter.
text	string	toLowerCase()	Supplies the text to convert to lowercase.
c	unsigned char	Lambda inside toLowerCase()	Receives each character for lowercase conversion.
text	string	normalizeSpaces()	Supplies the text whose whitespace is normalized.
command	string	processCommand()	Supplies the command to be interpreted.
direction	string	movePlayer()	Supplies the requested movement direction.
subject	string	getQuestion()	Identifies the question bank to use.
number	int	getQuestion()	Supplies the question number, which is reduced using

			number % 5 to select an array entry.
question	Question	askQuestion()	Supplies the question, answer options, correct answer, and hint.
finalQuestion	bool	askQuestion()	Indicates whether the current question prohibits artifact use. Its default argument is false.
subject	string	guardianBattle()	Identifies the guardian and associated educational subject.
page	int	discoverJournal()	Identifies the journal page to display and mark as discovered.

VI. Rules Governing the Data

Data Item	**Rule Implemented in the Code**
rank	Beginner: below 100 XP; Apprentice: 100–199 XP; Scholar: 200–299 XP; Master: 300 XP or higher.
currentRoom	Uses "Main Hall", "Library", "Hall of Numbers", "Laboratory", "Server Room", and "Sealed Chamber" as movement locations.

hasKey	Becomes true when the Library key is collected and false when it is used to open the chest.
libraryKeyTaken	Remains true after key collection, preventing the key from being collected again.
chestOpened	Becomes true after the chest is opened, preventing repeated opening.
Artifact possession flags	Become false when their respective artifacts are used. All four are also set to false before the final question.
successful	Becomes true after a correct answer or a Time Crystal skip. Both outcomes cause opponent damage.
xp and correct	Increase after a correct answer. A Time Crystal skip increases neither value.
incorrect and hp	An incorrect answer increases incorrect by one and decreases hp by 20.
guardiansDefeated	Increases by one after a guardian is defeated. Access to the Sealed Chamber requires a value of four.
headmasterHP	It is reduced during the main battle but is kept at a minimum of 20. A correct final answer is called victoryEnding(); the code does not explicitly assign zero to this variable.

SYSTEM SPECIFICATION

The Last Trial: Secrets of Blackwood Academy is a console-based educational adventure game developed using C++. Players explore interconnected rooms through text commands, collect items, and answer educational questions to defeat four guardians. Completing the guardian challenges unlocks the Sealed Chamber, where the player faces the Headmaster's final trial.

I. SOFTWARE SPECIFICATIONS

1. Programming Language

- The program uses C++ and requires a compiler supporting the C++11 standard or later.

2. Application Type

- The system is a console or command-line application with a text-based interface.

3. Program Components

- The program includes player initialization, command processing, room navigation, map display, inventory management, educational questions, guardian battles, the Headmaster battle, rank progression, saving, loading, and victory and defeat handling.

4. Input Method

- The program accepts keyboard input through complete text lines. Inputs include the player's name, movement and action commands, answer choices A–D, artifact selections, and menu responses.

5. Output Method

- The program displays text and ASCII art, including the story, room descriptions, map, available actions, player statistics, inventory, questions, health bars, rewards, and ending messages.

6. File Access

- Saving uses blackwood_save.txt, blackwood_save.tmp, and blackwood_save.bak. The working directory must permit file creation, writing, deletion, and renaming for all save operations to succeed.

II. FUNCTIONAL SPECIFICATIONS

1. Player Initialization and Startup

- The program initializes the player with 100 HP, 0 XP, Beginner rank, and the Main Hall as the starting location. It checks for an existing save and offers to load it. If no save is loaded, it requests a nonblank name and displays the introduction and mechanics.

2. Command Processing

- The system recognizes “go north,” “go south,” “go east,” “go west,” “look,” “map,” “take key,” “open chest,” “challenge,” “inventory,” “status,” “help,” “save,” “load,” and “quit.” Input is normalized for capitalization and whitespace. Empty and unknown commands produce appropriate messages.

3. Exploration and Movement

- The Main Hall connects north to the Laboratory, south to the Server Room, east to the Hall of Numbers, and west to the Library. The Library, Hall of Numbers, and Laboratory connect back to the Main Hall. The Server Room leads south to the Sealed Chamber only after all four guardians are defeated.

4. Map and Information Display

- The map displays the academy layout, current position, guardian states, key and chest availability, and chamber access. The look command displays the current room's description. Inventory and status commands display items, seals, player statistics, and guardian progress.

5. Key and Chest System

- The Library key can be collected once. Opening the chest in the Hall of Numbers requires and consumes the key. The chest can be opened once and awards the Oracle's Coin.

6. Educational Questions

- The game contains 20 guardian-bank questions, nine separate Headmaster questions, and one final question. Questions cover English, Mathematics, Science, and Computer subjects. Each question has four choices and allows up to three attempts per call. Question banks follow a fixed sequence and cycle when additional questions are required.

7. Answer Evaluation

- A correct response awards 20 XP and causes the battle function to deduct 20 opponent HP. A wrong valid response deducts 20 player HP, crosses out the selected choice, and consumes one

attempt. Invalid input and already-crossed-out choices do not consume attempts. When attempts run out while the player remains alive, the correct answer is displayed.

8. Inventory and Artifacts

- The system tracks the key and four artifacts. Time Crystal skips a question without XP or opponent damage. Rune of Elimination removes one available wrong choice. Oracle's Coin removes up to two available wrong choices. Wisdom Scroll displays a hint. Successful use consumes the artifact. Artifacts are cleared and prohibited before the final question.

9. Guardians and Rewards

- The Archivist occupies the Library, The Numeral the Hall of Numbers, Dr. Vex the Laboratory, and The Glitch the Server Room. Each guardian starts with 100 HP and requires five correct responses to defeat.

Guardian victory records completion, marks the corresponding seal as collected, and displays a journal passage. The Archivist awards the Wisdom Scroll, The Numeral the Rune of Elimination, and Dr. Vex the Time Crystal. The Glitch awards the Oracle's Coin only if the chest has not been opened.

10. Rank Progression

- Ranks are Beginner at 0–99 XP, Apprentice at 100–199 XP, Scholar at 200–299 XP, and Master at 300 XP or higher. Rank updates prevent downgrading during normal play. Loading reconstructs rank from the saved XP.

11. Final Trial

- Defeating all four guardians unlocks the Sealed Chamber. Entering it starts the Headmaster battle. The Headmaster begins with 200 HP, and nine correct responses reduce this to 20 HP. A separate final question then triggers victory when answered correctly.

12. Defeat and Continuation

- At zero player HP, the system displays a START OVER or QUIT menu. START OVER requires the same name, compared without case sensitivity, and restores HP to 100 while retaining accumulated player progress. An unfinished battle restarts because opponent HP and question position are not saved.

13. Saving, Loading, and Exit

- The system supports manual saving and loading. It attempts an automatic save after each command for which processCommand() returns true. Saving uses a temporary file and backs up an existing main save before replacement. Loading validates stored data before replacing the current player. File errors are reported. The exploration quit command offers an optional save before exiting.

III. DATA AND STATE REQUIREMENTS

The Player structure stores the player's name, current room, rank, HP, XP, correct and incorrect response counts, key and chest states, four artifact flags, and four defeated-guardian flags.

Guardian count and collected-seal displays are derived from the defeated-guardian flags. Journal passages are displayed without separate persistent journal-page flags.The Question structure stores question text,

four answer options, an integer correct-answer index from 0 to 3, and a hint. Opponent HP, question position, and phase flags are local battle variables. Save files do not preserve unfinished battle state or a final-victory completion flag.

IV. SYSTEM FLOW AND GAMEPLAY REQUIREMENTS

The program initializes the player, displays the title, and offers to load saved progress. A new game displays the introduction and instructions; a loaded game restores validated player information.

The command loop supports exploration, item interaction, information display, and guardian battles. Completing all four guardian challenges unlocks the final chamber. Defeat offers continuation or quitting, while a correct final answer displays victory.

The program reports file-operation failures. Progress is preserved between sessions only when saving succeeds.

HARDWARE SPECIFICATION

DEVICE	Laptop
BRAND AND MODEL	Lenovo V16 G4 IRU 
PROCESSOR	Intel i5-13420H 2.1GHz
MEMORY (RAM)	8GB
STORAGE	256GB SSD
DISPLAY	15.6" FHD